

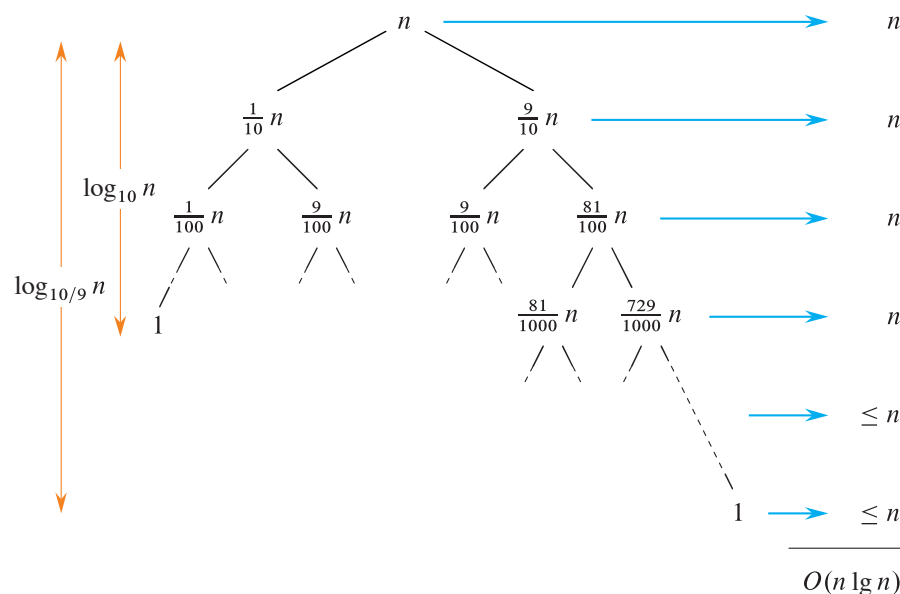


Figure 7.4 A recursion tree for QUICKSORT in which PARTITION always produces a 9-to-1 split, yielding a running time of $O(n \lg n)$. Nodes show subproblem sizes, with per-level costs on the right.

at most n . The recursion terminates at depth $\log_{10/9} n = \Theta(\lg n)$. Thus, with a 9-to-1 proportional split at every level of recursion, which intuitively seems highly unbalanced, quicksort runs in $O(n \lg n)$ time—asymptotically the same as if the split were right down the middle. Indeed, even a 99-to-1 split yields an $O(n \lg n)$ running time. In fact, any split of *constant* proportionality yields a recursion tree of depth $\Theta(\lg n)$, where the cost at each level is $O(n)$. The running time is therefore $O(n \lg n)$ whenever the split has constant proportionality. The ratio of the split affects only the constant hidden in the O -notation.

Intuition for the average case

To develop a clear notion of the expected behavior of quicksort, we must assume something about how its inputs are distributed. Because quicksort determines the sorted order using only comparisons between input elements, its behavior depends on the relative ordering of the values in the array elements given as the input, not on the particular values in the array. As in the probabilistic analysis of the hiring problem in Section 5.2, assume that all permutations of the input numbers are equally likely and that the elements are distinct.

When quicksort runs on a random input array, the partitioning is highly unlikely to happen in the same way at every level, as our informal analysis has assumed.

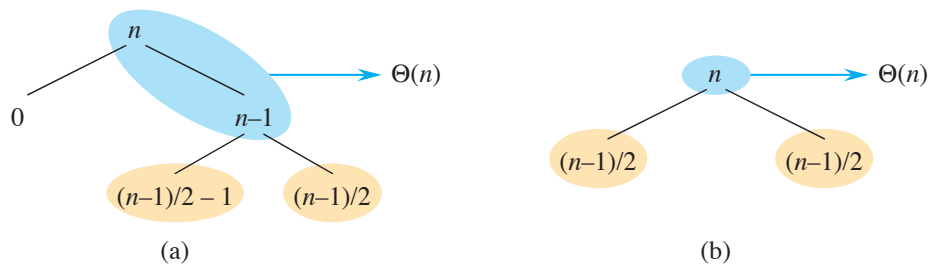


Figure 7.5 (a) Two levels of a recursion tree for quicksort. The partitioning at the root costs n and produces a “bad” split: two subarrays of sizes 0 and $n - 1$. The partitioning of the subarray of size $n - 1$ costs $n - 1$ and produces a “good” split: subarrays of size $(n - 1)/2 - 1$ and $(n - 1)/2$. (b) A single level of a recursion tree that is well balanced. In both parts, the partitioning cost for the subproblems shown with blue shading is $\Theta(n)$. Yet the subproblems remaining to be solved in (a), shown with tan shading, are no larger than the corresponding subproblems remaining to be solved in (b).

We expect that some of the splits will be reasonably well balanced and that some will be fairly unbalanced. For example, Exercise 7.2-6 asks you to show that about 80% of the time PARTITION produces a split that is at least as balanced as 9 to 1, and about 20% of the time it produces a split that is less balanced than 9 to 1.

In the average case, PARTITION produces a mix of “good” and “bad” splits. In a recursion tree for an average-case execution of PARTITION, the good and bad splits are distributed randomly throughout the tree. Suppose for the sake of intuition that the good and bad splits alternate levels in the tree, and that the good splits are best-case splits and the bad splits are worst-case splits. Figure 7.5(a) shows the splits at two consecutive levels in the recursion tree. At the root of the tree, the cost is n for partitioning, and the subarrays produced have sizes $n - 1$ and 0: the worst case. At the next level, the subarray of size $n - 1$ undergoes best-case partitioning into subarrays of size $(n - 1)/2 - 1$ and $(n - 1)/2$. Let’s assume that the base-case cost is 1 for the subarray of size 0.

The combination of the bad split followed by the good split produces three subarrays of sizes 0, $(n - 1)/2 - 1$, and $(n - 1)/2$ at a combined partitioning cost of $\Theta(n) + \Theta(n - 1) = \Theta(n)$. This situation is at most a constant factor worse than that in Figure 7.5(b), namely, where a single level of partitioning produces two subarrays of size $(n - 1)/2$, at a cost of $\Theta(n)$. Yet this latter situation is balanced! Intuitively, the $\Theta(n - 1)$ cost of the bad split in Figure 7.5(a) can be absorbed into the $\Theta(n)$ cost of the good split, and the resulting split is good. Thus, the running time of quicksort, when levels alternate between good and bad splits, is like the running time for good splits alone: still $O(n \lg n)$, but with a slightly larger constant hidden by the O -notation. We’ll analyze the expected running time of a randomized version of quicksort rigorously in Section 7.4.2.

Exercises**7.2-1**

Use the substitution method to prove that the recurrence $T(n) = T(n-1) + \Theta(n)$ has the solution $T(n) = \Theta(n^2)$, as claimed at the beginning of Section 7.2.

7.2-2

What is the running time of QUICKSORT when all elements of array A have the same value?

7.2-3

Show that the running time of QUICKSORT is $\Theta(n^2)$ when the array A contains distinct elements and is sorted in decreasing order.

7.2-4

Banks often record transactions on an account in order of the times of the transactions, but many people like to receive their bank statements with checks listed in order by check number. People usually write checks in order by check number, and merchants usually cash them with reasonable dispatch. The problem of converting time-of-transaction ordering to check-number ordering is therefore the problem of sorting almost-sorted input. Explain persuasively why the procedure INSERTION-SORT might tend to beat the procedure QUICKSORT on this problem.

7.2-5

Suppose that the splits at every level of quicksort are in the constant proportion α to β , where $\alpha + \beta = 1$ and $0 < \alpha \leq \beta < 1$. Show that the minimum depth of a leaf in the recursion tree is approximately $\log_{1/\alpha} n$ and that the maximum depth is approximately $\log_{1/\beta} n$. (Don't worry about integer round-off.)

7.2-6

Consider an array with distinct elements and for which all permutations of the elements are equally likely. Argue that for any constant $0 < \alpha \leq 1/2$, the probability is approximately $1 - 2\alpha$ that PARTITION produces a split at least as balanced as $1 - \alpha$ to α .

7.3 A randomized version of quicksort

In exploring the average-case behavior of quicksort, we have assumed that all permutations of the input numbers are equally likely. This assumption does not always hold, however, as, for example, in the situation laid out in the premise for

Exercise 7.2-4. Section 5.3 showed that judicious randomization can sometimes be added to an algorithm to obtain good expected performance over all inputs. For quicksort, randomization yields a fast and practical algorithm. Many software libraries provide a randomized version of quicksort as their algorithm of choice for sorting large data sets.

In Section 5.3, the RANDOMIZED-HIRE-ASSISTANT procedure explicitly permutes its input and then runs the deterministic HIRE-ASSISTANT procedure. We could do the same for quicksort as well, but a different randomization technique yields a simpler analysis. Instead of always using $A[r]$ as the pivot, a randomized version randomly chooses the pivot from the subarray $A[p : r]$, where each element in $A[p : r]$ has an equal probability of being chosen. It then exchanges that element with $A[r]$ before partitioning. Because the pivot is chosen randomly, we expect the split of the input array to be reasonably well balanced on average.

The changes to PARTITION and QUICKSORT are small. The new partitioning procedure, RANDOMIZED-PARTITION, simply swaps before performing the partitioning. The new quicksort procedure, RANDOMIZED-QUICKSORT, calls RANDOMIZED-PARTITION instead of PARTITION. We'll analyze this algorithm in the next section.

RANDOMIZED-PARTITION(A, p, r)

```

1   $i = \text{RANDOM}(p, r)$ 
2  exchange  $A[r]$  with  $A[i]$ 
3  return PARTITION( $A, p, r$ )

```

RANDOMIZED-QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3      RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4      RANDOMIZED-QUICKSORT( $A, q + 1, r$ )

```

Exercises

7.3-1

Why do we analyze the expected running time of a randomized algorithm and not its worst-case running time?

7.3-2

When RANDOMIZED-QUICKSORT runs, how many calls are made to the random-number generator RANDOM in the worst case? How about in the best case? Give your answer in terms of Θ -notation.

7.4 Analysis of quicksort

Section 7.2 gave some intuition for the worst-case behavior of quicksort and for why we expect the algorithm to run quickly. This section analyzes the behavior of quicksort more rigorously. We begin with a worst-case analysis, which applies to either QUICKSORT or RANDOMIZED-QUICKSORT, and conclude with an analysis of the expected running time of RANDOMIZED-QUICKSORT.

7.4.1 Worst-case analysis

We saw in Section 7.2 that a worst-case split at every level of recursion in quicksort produces a $\Theta(n^2)$ running time, which, intuitively, is the worst-case running time of the algorithm. We now prove this assertion.

We'll use the substitution method (see Section 4.3) to show that the running time of quicksort is $O(n^2)$. Let $T(n)$ be the worst-case time for the procedure QUICKSORT on an input of size n . Because the procedure PARTITION produces two subproblems with total size $n - 1$, we obtain the recurrence

$$T(n) = \max \{T(q) + T(n - 1 - q) : 0 \leq q \leq n - 1\} + \Theta(n), \quad (7.1)$$

We guess that $T(n) \leq cn^2$ for some constant $c > 0$. Substituting this guess into recurrence (7.1) yields

$$\begin{aligned} T(n) &\leq \max \{cq^2 + c(n - 1 - q)^2 : 0 \leq q \leq n - 1\} + \Theta(n) \\ &= c \cdot \max \{q^2 + (n - 1 - q)^2 : 0 \leq q \leq n - 1\} + \Theta(n). \end{aligned}$$

Let's focus our attention on the maximization. For $q = 0, 1, \dots, n - 1$, we have

$$\begin{aligned} q^2 + (n - 1 - q)^2 &= q^2 + (n - 1)^2 - 2q(n - 1) + q^2 \\ &= (n - 1)^2 + 2q(q - (n - 1)) \\ &\leq (n - 1)^2 \end{aligned}$$

because $q \leq n - 1$ implies that $2q(q - (n - 1)) \leq 0$. Thus every term in the maximization is bounded by $(n - 1)^2$.

Continuing with our analysis of $T(n)$, we obtain

$$\begin{aligned}
T(n) &\leq c(n-1)^2 + \Theta(n) \\
&\leq cn^2 - c(2n-1) + \Theta(n) \\
&\leq cn^2,
\end{aligned}$$

by picking the constant c large enough that the $c(2n-1)$ term dominates the $\Theta(n)$ term. Thus $T(n) = O(n^2)$. Section 7.2 showed a specific case where quicksort takes $\Omega(n^2)$ time: when partitioning is maximally unbalanced. Thus, the worst-case running time of quicksort is $\Theta(n^2)$.

7.4.2 Expected running time

We have already seen the intuition behind why the expected running time of RANDOMIZED-QUICKSORT is $O(n \lg n)$: if, in each level of recursion, the split induced by RANDOMIZED-PARTITION puts any constant fraction of the elements on one side of the partition, then the recursion tree has depth $\Theta(\lg n)$ and $O(n)$ work is performed at each level. Even if we add a few new levels with the most unbalanced split possible between these levels, the total time remains $O(n \lg n)$. We can analyze the expected running time of RANDOMIZED-QUICKSORT precisely by first understanding how the partitioning procedure operates and then using this understanding to derive an $O(n \lg n)$ bound on the expected running time. This upper bound on the expected running time, combined with the $\Theta(n \lg n)$ best-case bound we saw in Section 7.2, yields a $\Theta(n \lg n)$ expected running time. We assume throughout that the values of the elements being sorted are distinct.

Running time and comparisons

The QUICKSORT and RANDOMIZED-QUICKSORT procedures differ only in how they select pivot elements. They are the same in all other respects. We can therefore analyze RANDOMIZED-QUICKSORT by considering the QUICKSORT and PARTITION procedures, but with the assumption that pivot elements are selected randomly from the subarray passed to RANDOMIZED-PARTITION. Let's start by relating the asymptotic running time of QUICKSORT to the number of times elements are compared (all in line 4 of PARTITION), understanding that this analysis also applies to RANDOMIZED-QUICKSORT. Note that we are counting the number of times that *array elements* are compared, not comparisons of indices.

Lemma 7.1

The running time of QUICKSORT on an n -element array is $O(n + X)$, where X is the number of element comparisons performed.

Proof The running time of QUICKSORT is dominated by the time spent in the PARTITION procedure. Each time PARTITION is called, it selects a pivot element, which is never included in any future recursive calls to QUICKSORT and PARTITION. Thus, there can be at most n calls to PARTITION over the entire execution of the quicksort algorithm. Each time QUICKSORT calls PARTITION, it also recursively calls itself twice, so there are at most $2n$ calls to the QUICKSORT procedure itself.

One call to PARTITION takes $O(1)$ time plus an amount of time that is proportional to the number of iterations of the **for** loop in lines 3–6. Each iteration of this **for** loop performs one comparison in line 4, comparing the pivot element to another element of the array A . Therefore, the total time spent in the **for** loop across all executions is proportional to X . Since there are at most n calls to PARTITION and the time spent outside the **for** loop is $O(1)$ for each call, the total time spent in PARTITION outside of the **for** loop is $O(n)$. Thus the total time for quicksort is $O(n + X)$. ■

Our goal for analyzing RANDOMIZED-QUICKSORT, therefore, is to compute the expected value $E[X]$ of the random variable X denoting the total number of comparisons performed in all calls to PARTITION. To do so, we must understand when the quicksort algorithm compares two elements of the array and when it does not. For ease of analysis, let's index the elements of the array A by their position in the sorted output, rather than their position in the input. That is, although the elements in A may start out in any order, we'll refer to them by $z_1, z_2, \dots, z_n$, where $z_1 < z_2 < \dots < z_n$, with strict inequality because we assume that all elements are distinct. We denote the set $\{z_i, z_{i+1}, \dots, z_j\}$ by Z_{ij} .

The next lemma characterizes when two elements are compared.

Lemma 7.2

During the execution of RANDOMIZED-QUICKSORT on an array of n distinct elements $z_1 < z_2 < \dots < z_n$, an element z_i is compared with an element z_j , where $i < j$, if and only if one of them is chosen as a pivot before any other element in the set Z_{ij} . Moreover, no two elements are ever compared twice.

Proof Let's look at the first time that an element $x \in Z_{ij}$ is chosen as a pivot during the execution of the algorithm. There are three cases to consider. If x is neither z_i nor z_j —that is, $z_i < x < z_j$ —then z_i and z_j are not compared at any subsequent time, because they fall into different sides of the partition around x . If $x = z_i$, then PARTITION compares z_i with every other item in Z_{ij} . Similarly, if $x = z_j$, then PARTITION compares z_j with every other item in Z_{ij} . Thus, z_i and z_j are compared if and only if the first element to be chosen as a pivot from Z_{ij} is either z_i or z_j . In the latter two cases, where one of z_i and z_j is chosen

as a pivot, since the pivot is removed from future comparisons, it is never compared again with the other element. ■

As an example of this lemma, consider an input to quicksort of the numbers 1 through 10 in some arbitrary order. Suppose that the first pivot element is 7. Then the first call to PARTITION separates the numbers into two sets: $\{1, 2, 3, 4, 5, 6\}$ and $\{8, 9, 10\}$. In the process, the pivot element 7 is compared with all other elements, but no number from the first set (e.g., 2) is or ever will be compared with any number from the second set (e.g., 9). The values 7 and 9 are compared because 7 is the first item from $Z_{7,9}$ to be chosen as a pivot. In contrast, 2 and 9 are never compared because the first pivot element chosen from $Z_{2,9}$ is 7.

The next lemma gives the probability that two elements are compared.

Lemma 7.3

Consider an execution of the procedure RANDOMIZED-QUICKSORT on an array of n distinct elements $z_1 < z_2 < \dots < z_n$. Given two arbitrary elements z_i and z_j where $i < j$, the probability that they are compared is $2/(j - i + 1)$.

Proof Let's look at the tree of recursive calls that RANDOMIZED-QUICKSORT makes, and consider the sets of elements provided as input to each call. Initially, the root set contains all the elements of Z_{ij} , since the root set contains every element in A . The elements belonging to Z_{ij} all stay together for each recursive call of RANDOMIZED-QUICKSORT until PARTITION chooses some element $x \in Z_{ij}$ as a pivot. From that point on, the pivot x appears in no subsequent input set. The first time that RANDOMIZED-SELECT chooses a pivot $x \in Z_{ij}$ from a set containing all the elements of Z_{ij} , each element in Z_{ij} is equally likely to be x because the pivot is chosen uniformly at random. Since $|Z_{ij}| = j - i + 1$, the probability is $1/(j - i + 1)$ that any given element in Z_{ij} is the first pivot chosen from Z_{ij} . Thus, by Lemma 7.2, we have

$$\begin{aligned} \Pr\{z_i \text{ is compared with } z_j\} &= \Pr\{z_i \text{ or } z_j \text{ is the first pivot chosen from } Z_{ij}\} \\ &= \Pr\{z_i \text{ is the first pivot chosen from } Z_{ij}\} \\ &\quad + \Pr\{z_j \text{ is the first pivot chosen from } Z_{ij}\} \\ &= \frac{2}{j - i + 1}, \end{aligned}$$

where the second line follows from the first because the two events are mutually exclusive. ■

We can now complete the analysis of randomized quicksort.

Theorem 7.4

The expected running time of RANDOMIZED-QUICKSORT on an input of n distinct elements is $O(n \lg n)$.

Proof The analysis uses indicator random variables (see Section 5.2). Let the n distinct elements be $z_1 < z_2 < \cdots < z_n$, and for $1 \leq i < j \leq n$, define the indicator random variable $X_{ij} = I\{z_i \text{ is compared with } z_j\}$. From Lemma 7.2, each pair is compared at most once, and so we can express X as follows:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

By taking expectations of both sides and using linearity of expectation (equation (C.24) on page 1192) and Lemma 5.1 on page 130, we obtain

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] && \text{(by linearity of expectation)} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ is compared with } z_j\} && \text{(by Lemma 5.1)} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} && \text{(by Lemma 7.3).} \end{aligned}$$

We can evaluate this sum using a change of variables ($k = j - i$) and the bound on the harmonic series in equation (A.9) on page 1142:

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_{i=1}^{n-1} O(\lg n) \\ &= O(n \lg n). \end{aligned}$$

This bound and Lemma 7.1 allow us to conclude that the expected running time of RANDOMIZED-QUICKSORT is $O(n \lg n)$ (assuming that the element values are distinct). ■

Exercises

7.4-1

Show that the recurrence

$$T(n) = \max \{T(q) + T(n - q - 1) : 0 \leq q \leq n - 1\} + \Theta(n)$$

has a lower bound of $T(n) = \Omega(n^2)$.

7.4-2

Show that quicksort's best-case running time is $\Omega(n \lg n)$.

7.4-3

Show that the expression $q^2 + (n - q - 1)^2$ achieves its maximum value over $q = 0, 1, \dots, n - 1$ when $q = 0$ or $q = n - 1$.

7.4-4

Show that RANDOMIZED-QUICKSORT's expected running time is $\Omega(n \lg n)$.

7.4-5

Coarsening the recursion, as we did in Problem 2-1 for merge sort, is a common way to improve the running time of quicksort in practice. We modify the base case of the recursion so that if the array has fewer than k elements, the subarray is sorted by insertion sort, rather than by continued recursive calls to quicksort. Argue that the randomized version of this sorting algorithm runs in $O(nk + n \lg(n/k))$ expected time. How should you pick k , both in theory and in practice?

★ 7.4-6

Consider modifying the PARTITION procedure by randomly picking three elements from subarray $A[p : r]$ and partitioning about their median (the middle value of the three elements). Approximate the probability of getting worse than an α -to- $(1 - \alpha)$ split, as a function of α in the range $0 < \alpha < 1/2$.